

* a getgitgirl free framework

the friends framework

six ai agents run a marketing function between them, one per friends character. it covers what each one does, how the work moves between them, and how to build it without it turning into a mess.

13 pages · the files are on github · built with claude
@getgitgirl



why a **team**, not one mega-prompt

most people start with one giant prompt. research this, write that, in my voice, and post it. it holds up for about a week.

what breaks

the prompt gets long. rules for writing sit next to rules for research and start contradicting each other. the context fills with things that mattered three tasks ago. and when a post comes out wrong you're scrolling 600 lines to find where.

split it by job

each function gets its own agent, its own short instruction file, its own two or three tools. an agent that only researches is easy to write and easy to fix, because there's only one thing it can get wrong.

one operator in the middle

the agents don't call each other. hermes takes what ross found and hands it to phoebe with a brief. if any agent could trigger any other, one bad output would set off four more and you'd never find the first.

the friends names earn their keep

you remember what ross does without opening a diagram. he digs stuff up. monica runs the room. chandler complains but he can write.

the six, plus hermes and gunther

one job each. screenshot this. the rest of the guide is a page per agent.

one agent, one job. they don't talk to each other. hermes moves the work between them, gunther watches that it keeps moving.

ross · research

who to target, what got shared this week, what a competitor just changed. the reading everyone skips.

phoebe · strategy

picks what's worth saying and writes the brief. the one who'll push for the riskier angle.

chandler · copy

writes everything that goes out, then rewrites it so it stops sounding like a template.

rachel · design

makes it look like money. carousels, thumbnails, the deck, this pdf.

joey · outreach

cold messages, follow-ups, proposals, the warm-up video. gets a stranger onto a call.

monica · ops + dms

posts on schedule, answers dms, checks every claim. the last one to touch it.

hermes · the operator

passes work between the six, runs the timers, sends you one message a day with what needs you.

gunther · the watchdog

watches that the work is actually happening. pings you the second something stalls or the spend spikes.

the **five** rules every agent loads first

before context, before the job. an agent that hasn't loaded these doesn't run.

1 · never invent facts

every claim traces to a source in the input. no source, write “[not in sources]” or escalate. a fluent guess is worse than a visible hole.

2 · nothing external without the gate

no send, post or dm reaches anyone except through monica's queue and a human's yes. agents draft; a person ships.

3 · escalation is success

when the brief says escalate, escalating is finishing the task. format: what i was doing, what i hit, the options, which i'd pick.

4 · run the spec's own checks

if the output follows a voice guide or a checklist, run it and attach the pass/fail table. no table means unfinished.

5 · corrections are permanent

a human's correction goes in the changelog and never happens again. you tell an agent when it's wrong. you don't argue with it.

the asymmetry every agent carries

a held draft costs a day. a wrong one costs the reputation the whole business runs on. when a rule is unclear in the moment, hold it.

rules, context, and the skill

every agent loads the same two files, plus one that's just for its job. that last one is the skill.

1 · rules

what no agent is allowed to do. brand lines you don't cross, legal limits, the actions that always need a person. one file, every agent loads it. write it before you build anything.

2 · context

who you are, who you sell to, what you sell, why someone picks you. what you'd tell a new hire on day one. this is why chandler sounds like your company.

3 · the skill

a written procedure for one job, like ross's research method or chandler's voice pass, saved as a file (a SKILL.md, a prompt file, whatever your runner reads). it's reusable. two agents that both need a voice pass load the same skill, and one agent can load several. this is the only part that changes between agents.

rules and context are shared. skills are the specialism, and they stack. build up a library and each new agent gets faster to stand up.

* the friends framework · one agent, close up

chandler

copy

every skill file carries the judgement as two examples: the output you want, and the slop it produces instead if you don't spell it out.

excellent

slide 3 — move: name the real cause. copy: “every new chat is a new employee's first day. you keep hiring, day one, forever. you've explained ‘we don't use exclamation marks’ eleven times. it remembers none of them.”

voice-check: claim paid · antithesis count 0 · banned-list clean

mediocre, and it won't look wrong to you

slide 3: “context is everything! without proper context, your ai simply can't deliver the results you're looking for. it's not about better prompts — it's about better inputs. ready to transform your workflow?”

four failures: unpaid claim · the antithesis crutch · guru vocab (“transform”) · a rhetorical-question close the account never writes.

monica runs an adversarial layer

every external artifact hits the gauntlet before a human sees it. checks run in order, cheapest and most fatal first, and stop at the first one that fails.

every check is a scan, not a feel

source integrity · compliance screen · false-authority screen · voice-tell scan · promise ledger · example-per-slide · hook placement · recipient safety. each one a count or a lookup a model with no judgement can run.

a fail goes back to the author, named

monica never fixes and forwards. that trains failing agents to stay sloppy, and makes the checker the silent author of changes nobody reviewed.

the breach usually arrives as good advice

“a before/after gallery would convert brilliantly” is genuinely effective, and also restricted advertising. that's why it's a checklist and not a vibe.

guessing is banned near anything real

seven triggers stop the line. a missing fact, a price, two files that conflict, the same check failing twice, a regulator or a named person, no playbook, or “i'm sure but i can't show why”. that last feeling is the fabrication state.

how the work moves

three routes through the team. every arrow is hermes handing off. the agents don't pass to each other.

content



a signal becomes an angle, a draft, a scheduled post.

leads



a name becomes a warm lead, a pitch, a booked call.

inbound



monica reads every dm. real enquiries go to joey. the rest she logs.

gunther sits off to the side, watching that nothing's stuck. if you can't draw these arrows for your own team, the roles aren't clear enough yet. draw them first.

roles, agents and models aren't the same thing

the real knock on multi-agent systems. you build an elaborate machine that makes the same output at five times the cost. here's how to not do that.

a role is a job

research, strategy, copy. eight of them. this is what the guide has been describing.

an agent is a running thing

bound to a role. a cron job and a script, a node in an orchestrator, or just a scheduled model call. it doesn't have to be an always-on autonomous process.

a model is the llm behind it

one model serves several roles with different context. some roles want a different model or different tools. ross needs a scraper, phoebe needs your best reasoning.

so don't run eight processes

hermes can call a role like a function: load governance + context + rules + that skill + the task, run the model, write the output back, set the status. cheaper, simpler, same behaviour.

eight roles. maybe two or three actual agents. one or two models. keep those three counts separate in your head.

don't build six. build **one**.

six agents set up in a weekend is six things breaking on monday. do it in this order.

01 · start with whatever's eating your week

for most people that's chandler or ross. build one agent, give it the three files, use it for two weeks. don't touch anything else.

02 · add the second, and write the hand-off down

now you can feel the join. write exactly what agent one produces and what agent two expects. that spec is more work than either agent, and it matters more.

03 · at three agents, bring in hermes

three is where doing the routing yourself stops being free. add the operator and the daily message. any sooner and it's just one more thing to maintain.

04 · add the rest when the last one got boring

if an agent still surprises you, it isn't finished. add the next when the current one has stopped needing you.

most people stop at three. six is a whole agency. build for the work you have.

does it **beat** one good agent?

before you trust this over a single well-designed agent, prove it. the failure mode is real. you get the same mediocre output at five times the cost.

put the same twenty briefs through each. blind-rate the outputs. log the tokens from your router. count the reworks.

setup	quality 1-5	human min	tokens	reworks /20
one good agent	—	—	—	—
3 roles: research → strategy → copy	—	—	—	—
the full framework	—	—	—	—

if it doesn't move quality or cut your time, don't run it. that's the actual test.

five ways it goes wrong

the ones that show up over and over. all cheap to avoid up front, all a pain to fix later.

01 · too many agents

a new one every time you have an idea. now there are twelve half-built and no system. pick a number, merge anything that overlaps.

02 · no rules file

skipped because it's the boring part. then an agent says something you can't back, in your name. rules go first, not after it happens.

03 · agents calling agents

one agent triggering another directly. one break becomes five and you can't find the first. everything through hermes, logged.

04 · full auto on publish

no person on anything that posts or sends. you saved thirty seconds and took a risk you can't see. someone signs off on anything with your name on it.

05 · one model for the team

your best model on a scrape summary, or your cheapest on a strategy call. the model choice is most of your bill and most of your quality.

start here

this week.

step 01

name the job costing you the most hours. one job.

step 02

grab templates/agent-role.md from the repo. one page, plain words.

step 03

build it with its three files. run it two weeks before you add a second.

the whole thing

one agent, three files, a person on the publish button. add the rest later, or don't.